Testing 1, 2, 3 (on software testing and why it's interesting even to non-developers/non-coders)

Cite as: Martin Paul Eve, "Testing 1, 2, 3 (on software testing and why it's interesting even to non-developers/non-coders)", *eve.gd: Martin Paul Eve*, March 14, 2025, https://doi.org/10.59348/bnh0a-7cv52.

My days working at Knowledge Commons are highly varied. It's great. I get up in the morning and do some research work, writing my next book (OK, that isn't KC work, I know, but it's part of my day). The day itself will then involve some hands-on programming (we're re-writing the Profiles system), writing grant applications, giving talks about open practices, meeting with colleagues, debugging infrastructure, responding to user queries and feedback, communicating with the team, and even a bit of fun banter in the team channel along the way.

My current coding focus, though, is writing so-called "unit tests" for our next generation of software (the aforementioned Profiles system). In many ways, these are tedious but necessary. It's repetitive work and it doesn't produce anything exciting to show end-users. However, I guess I do think that software unit testing is really *interesting* even for non-coders to understand: because it's quite a simple way that you can learn how modern coding practices try to ensure software correctness, which affects absolutely everyone who uses a computer.

To understand what a "unit test" is, you need to know a TINY bit about programming. Consider this method (or "function", which is basically a set of commands for the computer, and which I will shortly explain):

```
def process_input(input):
        return "hello " + input
```

This is a really unexciting function called “process_input” (def means “define”). It has what we call a “parameter” called “input” (this could be any word, but I chose “input”). The function takes this variable that you pass in and returns “hello “ and then whatever is in “input”. So if we passed “Martin” as input, the function is supposed to give us “hello Martin” back.

This is such a simple function. I mean, what could be more basic in programming terms?

Yet there are lots of things that could go wrong. What if someone in future modifies the function by accident so that it says “goodbye” instead? We need to constrain this function, so that we can always be sure it is working as intended. So, we write things called “unit tests”, that “exercise” the function. That is, we try to anticipate all the circumstances that might arise and define the behavior that we expect from the function under each variation of “input”.

The unit tests for this function might look like this:

```
class TestProcessInput(unittest.TestCase):
    """Tests for the process_input function."""

    def test_process_input_with_string(self):
        """Test process_input with a standard string."""
        result = process_input("world")
        self.assertEqual(result, "hello world")

    def test_process_input_with_empty_string(self):
        """Test process_input with an empty string."""
        result = process_input("")
        self.assertEqual(result, "hello ")

    def test_process_input_with_number(self):
        """Test process_input with a number (which will be converted to string)."""
        result = process_input(42)
        self.assertEqual(result, "hello 42")

    def test_process_input_with_special_characters(self):
        """Test process_input with special characters."""
        result = process_input("!@#$%^")
        self.assertEqual(result, "hello !@#$%^")

    def test_process_input_with_unicode(self):
        """Test process_input with unicode characters."""
        result = process_input("世界")
        self.assertEqual(result, "hello 世界")

    def test_process_input_with_none(self):
        """Test process_input with None (should raise TypeError)."""
        with self.assertRaises(TypeError):
            process_input(None)
```

Blimey. That's a lot of testing for such a simple function. So what are we testing for?

First, we're looking for good callers who behave as we expect. If you pass a string (text) to the function, does it return the right value? This is what the test_process_input_with_string test does.

Then, we look at an empty string. If we pass nothing "" to the function, do we get "hello " back, without any extra text? That's what test_process_input_with_empty_string does.

What about if we pass a number? If we give the function 42, do we get “hello 42”? There’s what’s called an implicit conversion here: we convert a number to text and have to hope it works. So worth testing. That’s test_process_input_with_number.

What about special characters? Might they cause a problem? We probably don’t know. So test_process_input_with_special_characters exercises that possibility.

How about Unicode from non-English domains? test_process_input_with_unicode checks that we get “hello 世界” when we pass “世界” to input and that these special characters don’t break the processing.

Finally, there is a special value in Python (the programming language I am using) called None. It refers to nothing. Emptiness. The void. It’s not the same as a blank string (“”). It is a reference to null memory. test_process_input_with_none checks that if this happens, the code raises an error that can be handled by the caller (a TypeError).

What’s interesting about this is how we can constrain functionality using tests to make sure that, in future, functions behave as we expect them to. However, it’s also critical to note that even writing unit tests for the most basic of functions can take a REALLY long time. As long as writing the original code. So why bother adding this extra 50% of time on to an already lengthy process? Because when you have a large codebase, worked on by multiple people, a small change in a highly depended-upon function can cause ripple effects. We need to make sure that the code will always work as intended.

There’s also a system called “test-driven development” (TDD) where the *idea* is that you write the tests first (in a failing state), then write the code that makes the test work. It sounds brilliant, but it’s very hard actually to pull off. We don’t do full TDD at Knowledge Commons, but we do make extensive use of unit testing (and integration testing).

I just find it interesting how computers end up checking that computers are behaving correctly, but that, at the end of the day, unit tests are only as good as the imagination of their author. If you can’t imagine what could go wrong, it’s quite hard to test for it. Comprehensive unit tests are more than

exercising each line of code and hoping it works. You have to think of all the possibilities that MIGHT occur and break what you are doing, which requires an in-depth knowledge of the code and how your chosen language works.

If you are interested further, the tests I am working on and that prompted this post can be found on GitHub.